

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import mean_squared_error, r2_score # to evaluate the fit

```

```

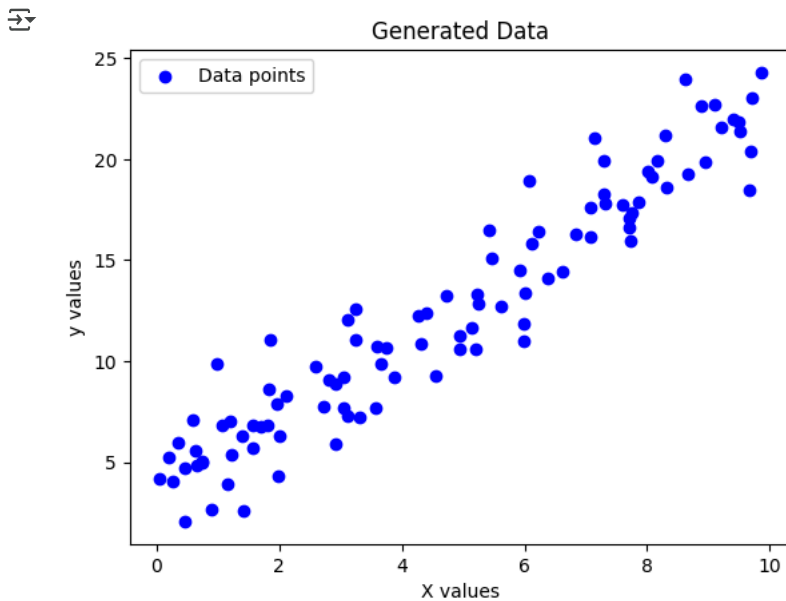
1 # Step 1: Generate synthetic data (y = 2x + 3 + noise)
2 np.random.seed(42)
3 X = np.random.rand(100, 1) * 10
4 y = 2 * X + 3 + np.random.randn(100, 1) * 2 # Add some noise

```

```

1 # Step 2: Visualize the data
2 plt.scatter(X, y, color='blue', label='Data points')
3 plt.title("Generated Data")
4 plt.xlabel("X values")
5 plt.ylabel("y values")
6 plt.legend()
7 plt.show()

```

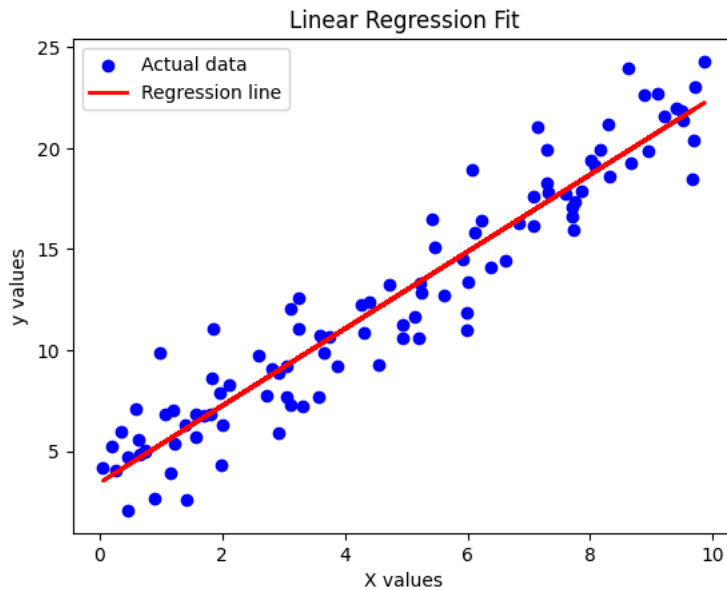


```

1 # Step 3: Train a Linear Regression model
2 model = LinearRegression()
3 model.fit(X, y) # Fits the model on the data
4
5 # Step 4: Predict using the model
6 y_pred = model.predict(X)
7
8 # Step 5: Evaluate the model
9 mse = mean_squared_error(y, y_pred)
10 r2 = r2_score(y, y_pred)
11
12 print(f"Model Coefficient (slope): {model.coef_[0][0]:.2f}")
13 print(f"Model Intercept: {model.intercept_[0]:.2f}")
14 print(f"Mean Squared Error: {mse:.2f}")
15 print(f"R-squared (R²) Score: {r2:.2f}")
16
17 # Step 6: Visualize the regression line
18 plt.scatter(X, y, color='blue', label='Actual data')
19 plt.plot(X, y_pred, color='red', linewidth=2, label='Regression line')
20 plt.title("Linear Regression Fit")
21 plt.xlabel("X values")
22 plt.ylabel("y values")
23 plt.legend()
24 plt.show()

```

Model Coefficient (slope): 1.91
 Model Intercept: 3.43
 Mean Squared Error: 3.23
 R-squared (R^2) Score: 0.91



1 # REAL WORLD EXAMPLE

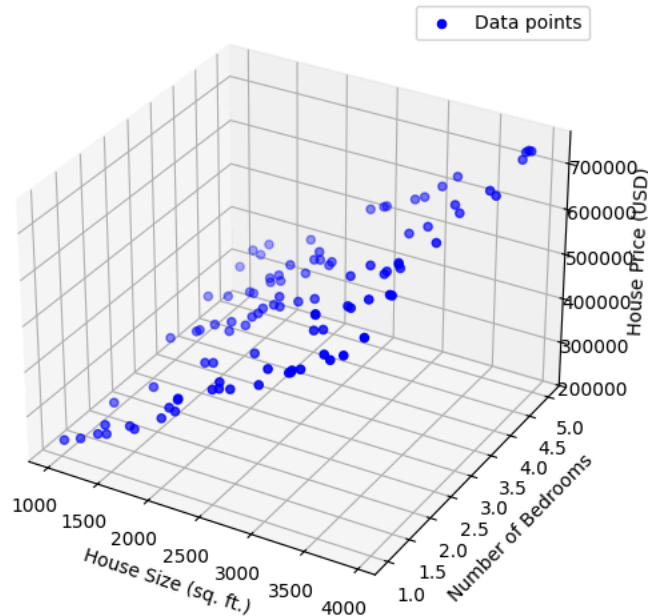
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import mean_squared_error, r2_score
5 from mpl_toolkits.mplot3d import Axes3D

1 # Step 1: Generate synthetic real estate data
2 np.random.seed(42) # Ensure reproducibility
3
4 # Generate house sizes (1000 - 4000 sq. ft.)
5 house_sizes = np.random.randint(1000, 4000, size=(100, 1))
6
7 # Generate number of bedrooms (1 - 5)
8 num_bedrooms = np.random.randint(1, 6, size=(100, 1))
9
10 # Formula: price = (150 * size) + (20000 * bedrooms) + base_price + noise
11 # We should get the coefficients close to this value
12 base_price = 50000
13 price_per_sqft = 150
14 price_per_bedroom = 20000
15
16 house_prices = (
17     price_per_sqft * house_sizes +
18     price_per_bedroom * num_bedrooms +
19     base_price +
20     np.random.randint(-20000, 20000, size=(100, 1))
21 )

1 # Combine the features into a single dataset
2 X = np.hstack((house_sizes, num_bedrooms)) # Combine both features into a single matrix
3 y = house_prices
4
5 # Step 2: Visualize data in 3D
6 fig = plt.figure(figsize=(10, 6))
7 ax = fig.add_subplot(111, projection='3d')
8 ax.scatter(house_sizes, num_bedrooms, house_prices, color='blue', label='Data points')
9 ax.set_xlabel("House Size (sq. ft.)")
10 ax.set_ylabel("Number of Bedrooms")
11 ax.set_zlabel("House Price (USD)")
12 plt.title("House Prices vs Size and Bedrooms")
13 plt.legend()
14 plt.show()
```



House Prices vs Size and Bedrooms



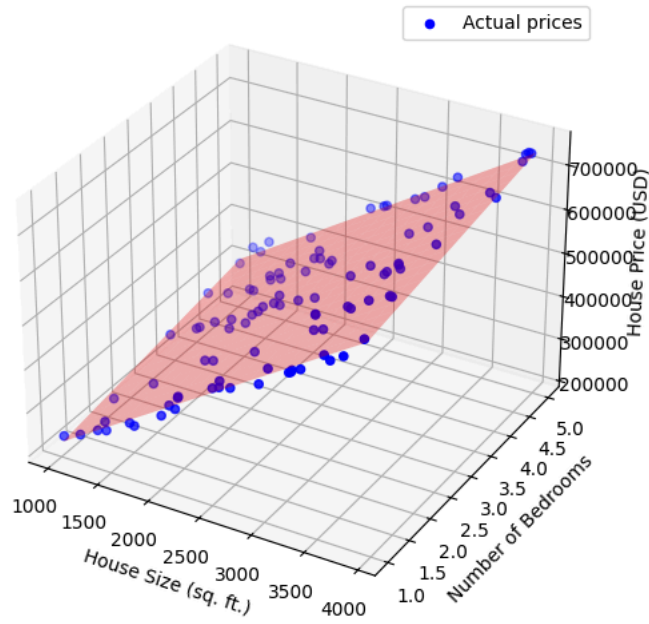
```

1 # Step 3: Train a Linear Regression model
2 model = LinearRegression()
3 model.fit(X, y)
4
5 # Step 4: Make predictions
6 predicted_prices = model.predict(X)
7
8 # Step 5: Evaluate the model
9 mse = mean_squared_error(y, predicted_prices)
10 r2 = r2_score(y, predicted_prices)
11
12 print(f"Model Coefficients (Price per Sq.Ft, Price per Bedroom): {model.coef_[0][0]:.2f}, {model.coef_[0][1]:.2f}")
13 print(f"Model Intercept (Base Price): ${model.intercept_[0]:.2f}")
14 print(f"Mean Squared Error: {mse:.2f}")
15 print(f"R-squared (R²) Score: {r2:.2f}")
16
17 # Step 6: Visualize the fit
18
19 fig = plt.figure(figsize=(10, 6))
20 ax = fig.add_subplot(111, projection='3d')
21
22 # Scatter plot of actual data points
23 ax.scatter(house_sizes, num_bedrooms, house_prices, color='blue', label="Actual prices")
24
25 # Generate grid for visualization
26 house_size_range = np.linspace(min(house_sizes), max(house_sizes), 20)
27 num_bedroom_range = np.linspace(min(num_bedrooms), max(num_bedrooms), 20)
28 house_size_grid, num_bedroom_grid = np.meshgrid(house_size_range, num_bedroom_range)
29
30 # Predict prices based on the grid
31 price_grid = (model.coef_[0][0] * house_size_grid +
32              model.coef_[0][1] * num_bedroom_grid +
33              model.intercept_)
34
35 # Plot the regression plane
36 ax.plot_surface(house_size_grid, num_bedroom_grid, price_grid, alpha=0.3, color='red')
37
38 # Set labels and title
39 ax.set_xlabel("House Size (sq. ft.)")
40 ax.set_ylabel("Number of Bedrooms")
41 ax.set_zlabel("House Price (USD)")
42 ax.set_title("Linear Regression Fit for House Prices")
43 ax.legend()
44
45 plt.show()
46
47

```

Model Coefficients (Price per Sq.Ft, Price per Bedroom): 147.99, 20262.80
Model Intercept (Base Price): \$52555.62
Mean Squared Error: 131017071.15
R-squared (R^2) Score: 0.99

Linear Regression Fit for House Prices



1 # Reasoning

2

3 # 1) Learned weights are close to the original price per sq. ft., price per bedroom

4 # 2) There is small variance because of limited samples and random noise added